

CURSO DE CIÊNCIA DA COMPUTAÇÃO
Disciplina: Computação Gráfica

**Removedor de Manchas em Imagens Digitais
com Python**

PROPOSTA DE PROJETO — AVALIAÇÃO A3

Nome do(a) Aluno(a) 1:	Igor Augusto Araujo Gê — Matrícula: 12823143203
Nome do(a) Aluno(a) 2:	Iago Fernandes Teixeira Rosado — Matrícula: 12824126703
Turma:	Ciências da Computação
Professor(a):	Karla Haryanna Santos Moura e Mikael Rosa de Medeiros
Semestre:	2026.1 Data de Entrega: 11/06/2026

1. Identificação do Projeto

Campo	Informação
Título do Projeto	Removedor de Manchas em Imagens Digitais com Python
Disciplina	Computação Gráfica
Curso	Ciência da Computação
Tipo de Avaliação	A3 — Projeto Prático Individual
Semestre	2026.1
Linguagem	Python 3.10+
Bibliotecas Principais	OpenCV · NumPy · scikit-image · Matplotlib
Plataforma	Linux / Windows (testado em Ubuntu 22.04 + WSL2)

2. Introdução e Contextualização

A preservação de documentos históricos e a restauração de imagens digitais são desafios recorrentes em arquivística, medicina e entretenimento. Manchas — resultantes de café, oxidação, umidade ou envelhecimento — degradam a legibilidade de textos manuscritos e impressos, dificultando sua digitalização e análise automatizada por OCR (Reconhecimento Óptico de Caracteres).

Este projeto de Avaliação A3 propõe o desenvolvimento de um sistema completo de detecção e remoção de manchas em imagens de documentos, explorando técnicas de Computação Gráfica com OpenCV e NumPy. A abordagem combina análise nos espaços de cor HSV e CIE L*a*b*, operações morfológicas e correção de cor global para remover manchas preservando o texto subjacente — algo que técnicas clássicas de inpainting não conseguem fazer de forma satisfatória quando a mancha cobre uma área extensa.

O estudante deverá demonstrar domínio sobre o pipeline completo de processamento de imagens: detecção baseada em desvio de cor, reconstrução por normalização cromática, remoção preservando texto e exportação de resultados com métricas quantitativas.

3. Objetivos

3.1 Objetivo Geral

Desenvolver um sistema capaz de detectar e remover automaticamente manchas em imagens de documentos digitalizados, preservando o texto e demais elementos gráficos originais, com validação quantitativa dos resultados.

3.2 Objetivos Específicos

- Implementar pipeline de detecção de manchas usando análise HSV/LAB com critérios multi-escala (manchas pequenas e grandes).

- Desenvolver algoritmo de correção de cor global que normaliza o tom da mancha em relação à cor estimada do papel.
- Preservar texto e elementos de alta frequência durante a remoção, evitando o borramento causado por inpainting clássico.
- Gerar relatório visual automatizado com imagens de cada etapa e métricas quantitativas (diff_manchas, cobertura).
- Permitir ajuste interativo da sensibilidade de detecção via parâmetro configurável pelo usuário.
- Validar o sistema em imagens reais de documentos manuscritos com manchas de café.

4. Fundamentação Teórica

4.1 Imagem Digital e Espaços de Cor

Uma imagem digital é uma representação matricial composta por pixels, cada um descrito por valores de intensidade ou cor. No modelo RGB (8 bits/canal), cada pixel cobre até 16,7 milhões de cores distintas. A conversão entre espaços de cor é operação fundamental no pré-processamento: o espaço **HSV** (Hue, Saturation, Value) separa matiz da luminosidade, facilitando a detecção de regiões com saturação anormal; o espaço **CIE L*a*b*** permite medir desvios de cromaticidade locais perceptualmente uniformes.

4.2 Técnicas de Processamento de Imagens

Técnica	Descrição	Aplicação no Projeto
Filtro Gaussiano	Suavização por convolução com kernel gaussiano; elimina texto, preserva cor de fundo	Estimativa do fundo da mancha
Morfologia Matemática	Erosão, dilatação, abertura e fechamento com elemento estruturante elíptico	Refino da máscara binária
Detecção de Bordas (LAB)	Desvio local entre pixel e sua vizinhança Gaussiana nos canais a* e b*	Detecção de manchas pequenas
Limiarização HSV	Limiar global de saturação para capturar regiões uniformemente coloridas	Detecção de manchas grandes
Inpainting TELEA	Propagação de cor por Fast Marching Method; eficaz em bordas finas (<2 px)	Suavização de borda da máscara

4.3 Correção de Cor por Normalização Global

A técnica central estima a cor real do papel a partir dos pixels não-manchados com maior luminância (percentil ≥ 70). Um fator de correção por canal BGR é calculado como $fator = cor_papel / cor_mancha_suave$, onde cor_mancha_suave é a imagem original filtrada com kernel Gaussiano grande (eliminando texto). O fator é aplicado pixel a pixel — preservando o texto porque tinta escura $\times$ fator ainda resulta em valor mais escuro que o papel corrigido.

5. Metodologia

5.1 Imagem de Teste

- Carta manuscrita digitalizada (*carta.jpg*) com manchas reais de café — resolução ~2400 × 3200 px, 8 bits por canal, modo RGB.
- Processamento realizado localmente em Python 3.10+ com OpenCV 4.x e NumPy.

5.2 Ambiente de Desenvolvimento

Componente	Especificação
Linguagem	Python 3.10 ou superior
Ambiente de Execução	IDE local (VSCode) / terminal Linux (Ubuntu 22.04 / WSL2)
Biblioteca Principal	OpenCV-Python (cv2) versão 4.x
Álgebra Matricial	NumPy 1.x
Análise de Cor/Forma	scikit-image 0.21+
Visualização	Matplotlib 3.x

5.3 Pipeline de Processamento

- **Detecção:** conversão HSV + $L^*a^*b^*$; desvio local (kernel 31×31) nos canais a^* e b^* ; limiar global de saturação; união dos mapas e refinamento morfológico.
- **Separação:** análise de componentes conectados; cálculo de área, raio, cor média e saturação por componente; exportação JSON.
- **Reconstrução:** estimativa da cor do papel (percentil ≥ 70); Gaussiana grande (kernel $\approx \min(h,w)/8$) para obter fundo; fator por canal [0.5, 6.0]; aplicação na região mascarada; suavização de borda com TELEA raio 2.
- **Exportação:** cinco imagens diagnóstico + JSON de metadados + log de métricas no terminal.

6. Estrutura de Implementação

6.1 Instalação das Dependências

```
pip install opencv-python numpy scikit-image matplotlib Pillow
```

6.2 Funções Principais

Função	Finalidade	Biblioteca
detectar_manchas(img, sensibilidade)	Detecção multi-critério HSV+LAB; retorna máscara binária e lista de componentes	OpenCV + scikit-image
reconstruir_imagem(img, mascara, modo)	Orquestra a reconstrução; chama <code>_corrigir_mancha_colorida</code> conforme o modo	OpenCV + NumPy

_corrigir_mancha_colorida(img, mascara)	Estimativa do papel, fator de correção, aplicação na região mascarada	NumPy
remover_manchas(img, mascara)	Aplica a máscara reconstruída; retorna imagem restaurada	NumPy
exportar_resultados(...)	Salva 5 imagens diagnóstico, JSON de metadados e métricas	OpenCV + Matplotlib
pipeline(caminho, sensibilidade, modo)	Função de alto nível que encadeia todas as etapas	—

7. Resultados Esperados

Ao final da execução, o sistema deverá produzir os seguintes artefatos visuais e analíticos:

- **01_deteccao.png** — Máscara binária das manchas com contornos vermelhos sobre a imagem original.
- **02_separacao.png** — Componentes conectados individualizados com cores distintas e bounding boxes.
- **03_reconstrucao.png** — Região reconstruída isolada, mostrando a correção de cor aplicada.
- **04_remocao.png** — Imagem final restaurada com manchas removidas e texto preservado.
- **05_antes_depois.png** — Comparativo lado a lado (original × restaurado) para avaliação visual.
- **manchas_detectadas.json** — Metadados por componente: área, raio, cor média, saturação, diff.

Tabela de métricas esperadas:

Métrica	Critério
diff_manchas	Diferença média de pixel na região da mancha. Meta: > 40 (referência obtida: ~45.65)
diff_resto	Diferença fora da máscara. Deve ser 0.00 — nenhum pixel externo é modificado
Cobertura (%)	Percentual da imagem coberto por manchas. Referência: ~5% na imagem de teste
N. componentes	Número de manchas detectadas. Referência: ~52 com sensibilidade = 1.5

8. Cronograma de Desenvolvimento

Fase / Atividade	Prazo Sugerido	Responsável
------------------	----------------	-------------

Levantamento bibliográfico e estudo de técnicas HSV/LAB	Semana 1	Aluno
Implementação do detector multi-critério (HSV + LAB)	Semana 1-2	Aluno
Implementação do pipeline de inpainting TELEA	Semana 2	Aluno
Desenvolvimento da correção de cor global (normalização)	Semana 3	Aluno
Testes e ajuste de parâmetros (sensibilidade)	Semana 3-4	Aluno
Geração automática de relatório visual (5 imagens)	Semana 4	Aluno
Avaliação quantitativa e ajuste fino	Semana 5	Aluno
Escrita do relatório técnico e entrega	Semana 6	Aluno

9. Critérios de Avaliação

Critério de Avaliação	Peso (%)	Observações
Documentação da origem da imagem	15%	Deve evidenciar participação ativa do aluno
Qualidade e variedade das técnicas	30%	Mínimo de 4 técnicas distintas implementadas
Correção e funcionamento do código	5%	Código deve executar sem erros
Análise crítica dos resultados	10%	Discussão técnica dos efeitos aplicados
Relatório técnico escrito	10%	Formatação, clareza e referências bibliográficas
Apresentação oral / defesa	30%	Domínio do conteúdo e clareza expositiva

10. Entregáveis

O aluno deverá submeter, em pasta compactada (.zip) ou repositório Git (GitHub/GitLab), os seguintes artefatos:

- **removedor_manchas.py** — Código-fonte principal com pipeline completo (detecção, reconstrução, remoção, exportação), comentado em português.
- **requirements.txt** — Lista de dependências Python com versões fixadas.
- **imagem_teste/** — Imagem(ns) utilizada(s) na validação (*carta.jpg* ou equivalente com manchas reais).
- **resultados/** — Pasta contendo as 5 imagens de diagnóstico e o JSON de metadados gerados na execução.
- **Proposta_A3_Igor_Augusto.pdf** — Este documento de proposta preenchido com os resultados reais obtidos.

- **README.md** — Instruções de execução: dependências, versões Python, parâmetros de linha de comando.

11. Referências Bibliográficas

- [1] GONZALEZ, R. C.; WOODS, R. E. *Processamento Digital de Imagens*. 3. ed. São Paulo: Pearson, 2010.
- [2] BRADSKI, G.; KAEHLER, A. *Learning OpenCV 4: Computer Vision with Python 3*. O'Reilly Media, 2019.
- [3] TELEA, A. An image inpainting technique based on the fast marching method. *Journal of Graphics Tools*, v. 9, n. 1, p. 23–34, 2004.
- [4] SZELISKI, R. *Computer Vision: Algorithms and Applications*. 2. ed. Springer, 2022.
- [5] OpenCV Documentation. Disponível em: <https://docs.opencv.org/> (acesso em 2026).
- [6] NumPy Documentation. Disponível em: <https://numpy.org/doc/> (acesso em 2026).
- [7] scikit-image. *Image processing in Python*. PeerJ, v. 2, e453, 2014.

12. Assinaturas e Validação

Assinatura do(a) Aluno(a) 1

Nome: Igor Augusto Araujo Gê — Mat. 12823143203

Data: 11/06/2026

Assinatura do(a) Aluno(a) 2

Nome: Iago Fernandes Teixeira Rosado — Mat. 12824126703

Data: 11/06/2026

Assinatura do(a) Professor(a) Orientador(a)

Nome: Karla Haryanna Santos Moura e Mikael Rosa de Medeiros

Data: 11/06/2026 Nota: _____